The Problem and Algorithms

Problem Definition

The single-source shortest-path problem involves finding the minimum total distance from a specified starting vertex to every other reachable vertex within a weighted graph. Dijkstra's algorithm is commonly used to solve this problem for graphs containing **non-negative** edge weights. It works by repeatedly selecting the vertex with the smallest known distance and examining its outgoing edges to identify shorter paths. The efficiency of this process depends partly on the priority queue used to manage the vertices. This project investigates two implementations of Dijkstra's algorithm, one using a binary min-heap and the other using a Fibonacci heap. By comparing these two approaches across graphs of different sizes and densities, the project aims to investigate how theoretical differences in time complexity translate into practical execution performance.

Binary Heap implementation

The graph was implemented in C++ using an adjacency list, where each vertex stores its outgoing edges and their associated weights. This approach was chosen to allow efficient access to neighbouring vertices without needing to search through the entire graph.

The binary min-heap was implemented through a custom BinaryMinHeap class using std::vector. Each HeapNode stores a vertex ID and its current shortest-known distance. The heap maintains the property that each parent node has a distance less than or equal to its children, ensuring that the vertex with the smallest distance remains at the root. Additionally, a position vector was used to track each vertex's current location within the heap.

The implementation consists of several key operations. The insert() function adds a new vertex and calls siftUp() to restore the heap order. Similarly, extractMin() removes the vertex with the smallest distance and uses siftDown() to maintain the heap structure. The decreaseKey() function updates a vertex's distance when Dijkstra's algorithm discovers a shorter route, while swapNodes() updates both the heap and position vector to ensure their locations remain consistent.

In terms of time complexity, insertion, extraction and decrease-key operations all have a worst-case complexity of $O(log\ V)$. The additional position vector allows a vertex's location within the heap to be found in $O(1)$ time. As a result, Dijkstra's algorithm using the binary heap has an overall time complexity of $O((V + E)log\ V)$, where (V) represents the number of vertices and (E) represents the number of edges. The heap itself requires $O(V)$ additional memory.

Fibonacci Heap Implementation

The Fibonacci heap was implemented in C++ as a collection of heap-ordered trees. Each node stores its key, parent and child pointers, links to its neighbouring nodes, a degree count and a mark flag. The root list and each child list use circular doubly linked lists, allowing nodes to be detached and joined without moving the remaining nodes. A minimum pointer keeps track of the smallest key. Dijkstra's algorithm retains a direct handle for each vertex, so decrease-key can access the required node without searching the heap.

The insert() operation adds a node to the root list and updates the minimum pointer. The extract_min() operation removes the smallest node, moves its children into the root list and consolidates trees with equal degrees. When two trees are joined, the root with the larger key becomes a child of the root with the smaller key. This restores a more compact structure after the minimum has been removed.

The decrease_key() operation lowers a node's key. If this breaks the order between the node and its parent, the node is cut from its parent and moved to the root list. A non-root node is marked after losing its first child. If it loses another child, it is also cut, and this process may continue upwards as a cascading cut. Nodes are owned through std::unique_ptr storage, while the parent, child and sibling pointers are non-owning links. Extracted nodes remain allocated but are marked inactive until the heap is destroyed, allowing invalid handles to be rejected safely.

In terms of theoretical complexity, insertion and decrease-key have O(1) amortised time complexity, while extract-min has O(log V) amortised time complexity. These are amortised bounds across a sequence of operations, rather than worst-case bounds for every individual call. Using the Fibonacci heap gives Dijkstra's algorithm an overall bound of O(E + V log V), with O(V) additional heap storage.

Purpose of the Comparison

The main purpose of this comparison is to investigate whether the Fibonacci heap's theoretically better time complexity translates into faster execution in practice. This is achieved by comparing the simpler array-based binary heap against the more complex, pointer-based Fibonacci heap. A wide range of datasets were used to explore how increases in graph size and density affect the relative performance of the two algorithms. Furthermore, the comparison investigates whether the Fibonacci heap's $O(1)$ amortised decrease-key operation provides a noticeable advantage over the binary heap's $O(log\ V)$ operation. Finally, the comparison aims to identify the practical trade-offs associated with a more complex implementation in terms of efficiency and computational overhead in real-world applications.

Theoretical Comparison and Predictions

Time Complexity Comparison

The complexity of both heaps has been broken down into the table below.

Operation	Binary Heap	Fibonacci Heap
Insert	O(log V)	O(1) amortised
Extract minimum	O(log V)	O(log V) amortised
Decrease key	O(log V)	O(1) amortised
Overall Dijkstra	**O((V + E) log V)**	**O(E + V log V) amortised**

In terms of insertion time complexity, the Fibonacci heap is theoretically more efficient, as the binary heap may need to move a new element upwards to restore heap order, while Fibonacci heaps can add nodes to their root list without immediately reorganising the structure. For the extract-min operation, both have the same asymptotic time complexity, although they achieve this differently. Binary heaps restore order using the siftDown() function, while Fibonacci heaps consolidate trees after removing the minimum element. For decrease-key operations, Fibonacci heaps again have a lower theoretical time complexity, as binary heaps may need to move an updated vertex upwards, while Fibonacci heaps use node cuts and cascading cuts to maintain their amortised performance guarantee. Overall, the potential restructuring required after insertions and decrease-key operations contributes to the binary heaps' higher theoretical time complexity compared to the Fibonacci heap.

Memory Usage and Implementation Overhead

Both the binary heap and Fibonacci heap require $O(V)$ memory, although they are implemented differently. Binary heaps use a contiguous array-based structure through std::vector to store HeapNode objects. Each HeapNode stores a vertex ID and its current distance, while an additional position vector is used to track vertex locations. Contiguous memory generally provides better cache locality, as nearby elements can be accessed more efficiently.

Fibonacci heaps use a more complex collection of interconnected trees, where nodes typically store additional pointers to their parents, children and siblings. While Fibonacci heaps also require $O(V)$ memory, they generally have greater memory overhead per node. Additionally, pointer-based structures can result in more scattered memory access, potentially reducing cache efficiency.

Binary heaps use relatively simple array indexing and swapping to maintain heap order, while Fibonacci heaps require more complex operations, including tree consolidation, node cuts and cascading cuts. Dynamic memory allocation and pointer management can introduce additional runtime overhead. These additional costs may outweigh the theoretical advantages of Fibonacci heaps, particularly for smaller graphs.

Impact of Graph Size and Density

The number of vertices (V) is one measure of graph size. As the number of vertices increases, Dijkstra's algorithm may need to process more vertices and maintain a larger priority queue. In both implementations, each reachable vertex is extracted once. Binary heap operations such as extractMin() and decreaseKey() can take $O(log\ V)$ time, meaning an increase in vertex count can increase the cost of these operations. Fibonacci heaps also have $O(log\ V)$ amortised extract-min operations, but their decrease-key operation has an $O(1)$ amortised time complexity.

Graph density describes how many connections exist relative to the number of possible connections. Increasing the number of edges (E) generally increases the number of connections Dijkstra needs to examine. More edges create additional opportunities for successful relaxations, potentially increasing the number of decrease-key operations. However, not every examined edge results in a shorter path, meaning decrease-key operations do not necessarily increase proportionally with the number of edges. Fibonacci heaps may benefit from workloads involving more decrease-key operations due to their better amortised time complexity.

Prediction provenance: the preserved AI-authored method note dated 1 November 2026 predates the recorded C++ performance measurements and proposes that indexed binary may win at these sizes, while Fibonacci may improve relatively with more successful updates. It explicitly does not record either student's personal prior beliefs. The predictions below are theory-based hypotheses; no independently dated student-authored prediction has been supplied, so this report does not present them as a verified record of personal expectations before testing.

Theoretical Performance Predictions

There are four main theoretical predictions considered in this comparison. Firstly, binary heaps are expected to perform better on smaller graphs. This is because binary heaps have a simpler array-based structure with less pointer management. Furthermore, their better cache locality and lower constant overhead may make operations faster in practice. For smaller graphs, the Fibonacci heap's better amortised time complexity may not provide enough benefit to offset its additional overhead.

The second prediction is that Fibonacci heaps may become more competitive as graph density increases. This is because more edges create additional opportunities for successful relaxations and decrease-key operations. Fibonacci heaps perform decrease-key in $O(1)$ amortised time, compared with $O(log\ V)$ for binary heaps. If sufficiently many decrease-key operations occur, their lower theoretical cost may become increasingly beneficial. However, additional edges do not necessarily product proportionally more decrease-key operations.

The third prediction is that increasing graph size may influence the performance difference. As the vertex count increases, the logarithmic cost of binary heap operations also grows. Fibonacci heaps retain their $O(1)$ amortised decrease-key complexity regardless of graph size. However, Fibonacci heaps still have $O(log\ V)$ amortised extract-min operations and potentially greater implementation overhead. Therefore, increasing graph size alone does not necessarily guarantee a performance advantage for Fibonacci heaps.

The final prediction is that a crossover point may exist. A crossover occurs when, under certain conditions, the Fibonacci heap begins outperforming the binary heap. This might occur on sufficiently large graphs with a high frequency of decrease-key operations. However, a crossover is not guaranteed within the tested dataset sizes and densities.

Expected Trade-offs

Binary heaps are relatively simple to implement, debug and maintain, while Fibonacci heaps require more complicated operations, such as tree consolidation and cascading cuts. Greater implementation complexity can introduce more opportunities for bugs and make ongoing maintenance more difficult.

Fibonacci heaps have better amortised time complexity for insertion and decrease-key operations. However, additional pointer management, memory allocations and structural maintenance can increase actual execution time. As a result, binary heaps may outperform Fibonacci heaps despite their less favourable theoretical bounds.

The practical advantage of each heap depends on the size and structure of the graph. Workloads with relatively few decrease-key operations may favour binary heaps, while workloads involving frequent decrease-key operations may provide more opportunities for fibonacci heaps to benefit from their theoretical advantage.

When selecting the appropriate algorithm, execution time, memory usage, implementation complexity and maintainability should all be considered. The theoretically more efficient algorithm is not automatically the most suitable option for every application.

Experimental Methodology

Dataset Design and Generation

A total of 27 graph datasets were used, covering three vertex counts: 250, 1,000 and 4,000. For each graph size, three edge-to-vertex ratios were tested: $E = 4V$, $E = 16V$ and $E = 64V$. Each configuration was represented by three datasets using different random seeds: 11, 22, and 33. The graphs were stored in CSV format, containing source, target and weight columns. Both algorithms process weighted graphs, with non-negative edge weights being a requirement of Dijkstra's algorithm. Using different graph sizes and edge counts allows us to investigate how the performance of each implementation changes as Dijkstra's workload increases.

Vertices	Sparse (4V)	Medium (16V)	Higher Density (64V)
250	1,000 edges	4,000 edges	16,000 edges
1,000	4,000 edges	16,000 edges	64,000 edges
4,000	16,000 edges	64,000 edges	256,000 edges

Original Independent Testing Environments

The binary heap was implemented in C++ using a custom BinaryMinHeap class with std::vector and a position lookup array. The implementation processes graph data from CSV files. The binary heap implementation was compiled using the C++17

standard and the GNU C++ compiler (g++). The -O2 optimisation flag was enabled to improve execution performance, while the -Wall and _Wextra flags were used to enable additional compiler warnings.

The hardware used for testing the binary heap implementation consisted of an AMD Ryzen 7 5700X 8-Core Processor with 32 GB of RAM, running windows 11.

The Fibonacci heap was implemented in C++ using a pointer-based structure consisting of a collection of heap-ordered trees. The implementation processes graph data from CSV files and uses the heap to manage vertices and their shortest-known distances during Dijkstra's algorithm. It was compiled using the C++17 standard and the GNU C++ compiler, GCC 14.2, with the -O2 optimisation flag enabled to improve execution performance.

Testing of the Fibonacci heap implementation was carried out in a shared Linux virtual machine.

Original Independent Benchmarking Procedures

The binary heap implementation used C++'s std::chrono::steady_clock to measure execution time, with timestamps recorded immediately before and after each execution of dijkstraBinary(). The elapsed time was then converted into microseconds (μs). To ensure the measurements focused on the algorithm itself, CSV reading, graph construction, printing results and collecting operation statistics were excluded from the timed section. This meant that the recorded times reflected the execution of Dijkstra's algorithm and its internal binary heap operations.

Before recording measurements, each dataset was run 10 times as a warm-up to reduce initial execution effects, such as cold caches. Dijkstra's algorithm was then executed 200 times per dataset, with each execution time recorded individually. Repeating the tests reduced reliance on a single measurement, which could be affected by background processes or CPU scheduling. The 200 recorded times were then sorted in ascending order, with the median calculated by averaging the two middle values. The median was chosen because it is less sensitive to occasional unusually slow executions than the arithmetic mean, providing a more representative measurement of typical execution time.

In addition to execution time, a separate run was performed to record heap insertions, extractions, decrease-key operations, examined edges and successful relaxations. These statistics were collected outside the timed runs to avoid the additional overhead of updating counters affecting the measurements. The results were then automatically exported to a CSV file for further analysis.

The Fibonacci heap implementation also used std::chrono::steady_clock, but its timing procedure differed from the binary heap's individual-run method. Each dataset

used vertex 0 as the source. Two untimed warm-up calls per algorithm were performed before eight measured batches. The batch size was fixed according to graph size: 32 calls for 250 vertices, eight calls for 1,000 vertices and two calls for 4,000 vertices. This produced 256, 64 and 16 timed calls per dataset respectively. Timed batches alternated with an assistant-reference binary heap to balance execution order. This reference was separate from James's binary heap implementation.

For each measured batch, the total elapsed time was divided by the number of calls to obtain a mean time per call. The reported execution time was the median of the eight batch means, expressed in microseconds. The first and third quartiles and the interquartile range were also retained in the detailed results file. Consequently, these values are medians of batch means, rather than medians of individually timed executions.

The timed section included initialisation, algorithm allocations, Dijkstra's algorithm, heap cleanup, destruction of the returned results and an update to a result sink. Graph parsing, correctness checks and output were excluded. Operation counts were collected separately from the timed calls. The recorded Linux run used a shared virtual machine without CPU pinning or control over background load, power mode or frequency scaling. Memory usage was not measured in this run. Because the two group implementations used different machines and timing procedures, their reported times cannot establish a controlled direct speed comparison.

Final matched comparison on Ben's computer — 4 November 2026

The final direct comparison was run on Ben's computer after the earlier independent tests used different machines. James's BinaryMinHeap and dijkstraBinary() implementation and Ben's Fibonacci heap and original Dijkstra template were compiled into the same x64 executable. The machine used Windows 11 Home (version 10.0.26200), an AMD Ryzen 5 7500F processor with 6 cores and 12 logical processors, and approximately 32 GB of RAM. Both implementations used MSVC 19.44.35223, C++17 and the /O2 and /DNDEBUG compiler flags. Their original source copies and both algorithm bodies were preserved. The earlier Windows and Linux environments describe exploratory measurements; the matched rerun provides the results used for the direct comparison. Recording clarification: James's reference in the video to different test machines concerns the earlier independent runs. The final comparative chart and reported ratios use the later matched rerun on Ben's computer.

All 27 core datasets used source vertex 0. After two untimed warm-up calls per implementation, eight paired trials were recorded for each graph, alternating which implementation ran first. Each batch contained 32, eight or two calls for 250, 1,000 or

4,000 vertices respectively. Parsing and conversion into each implementation's native graph representation took place before timing. Timing included initialisation, allocations, the algorithm, cleanup, returned-result destruction and a result-sink update. The median of eight batch means was calculated per instance, with inclusive quartiles and IQR retained. Each graph-family aggregate was the median of its three seed-specific medians. The run retained 432 raw batch measurements and 54 per-instance summaries.

James's binary implementation had the lower median runtime on all 27 tested instances, including all nine size and edge-ratio aggregates. At 4,000 vertices and E/V = 64, the aggregate medians were 1,331.150 µs for the binary implementation and 4,438.200 µs for the Fibonacci implementation, a Fibonacci/binary ratio of approximately 3.334. No crossover was observed within this tested workload.

These results compare the complete implementations rather than isolating heap choice. James inserts vertices when discovered, while Ben initially inserts every vertex. Ben also returns predecessor information, and the implementations use different integer types and tie-breaking policies. Background load, CPU affinity, power mode and frequency scaling remained uncontrolled, and memory was not measured. The result therefore does not establish that every binary heap is faster than every Fibonacci heap or disprove the Fibonacci heap's amortised bounds.

Fairness and Correctness Verification

To ensure a fair comparison, both implementations were benchmarked using the same 27 graph datasets, with vertex 0 as their starting vertex. Both implementations use Dijkstra's algorithm to solve the same single-source shortest-path problem and are expected to return the shortest distance from the starting vertex to every reachable vertex. By using identical graph data and starting vertices, the possibility of differences in graph structure influencing the comparison was reduced.

To verify correctness, the binary heap implementation was initially tested using smaller graphs with known shortest-path distances. Additional tests covered zero-weight edges, disconnected vertices and alternative starting vertices. Negative-weight edges were also tested to verify that invalid inputs were rejected. Heap operation statistics were used to check internal consistency, where the number of successful relaxations should equal the number of insertions, excluding the initial source insertion, plus the number of decrease-key operations.

The Fibonacci heap's test suite checks its internal structure as well as the shortest-path results. Its invariant checker verifies the links in each circular list, parent pointers, degree counts, heap order, minimum pointer and active-node membership. Randomised heap-operation tests call this checker after each operation. Additional tests explicitly trigger cascading cuts and check that foreign handles, extracted handles and attempts to increase a key are rejected. At the graph

level, the test suite compares complete distance vectors against an independent array-based Dijkstra reference and validates predecessor paths. Tests cover tied distances, zero-weight edges, disconnected graphs and large integer distances. The recorded source-0 Linux results file reports successful selftests, checks of all 27 core graphs and four supplied fixtures. These checks concern the Fibonacci implementation and its reference tests; they do not establish agreement with James's separately developed implementation.

The original independent runs used different hardware, compiler settings and benchmarking procedures, so their timings could not establish a controlled direct comparison. Agreement between the two submitted implementations had also not been verified at that stage. The matched rerun described below subsequently checked both complete distance vectors against the same oracle and used one machine and timing harness.

Matched-rerun correctness update — 4 November 2026
In the matched rerun, both complete distance vectors matched an independent array-based Dijkstra oracle on all 27 core graphs at source 0, four supplied fixtures and 80 additional fixed-seed random graphs. The fixtures also matched their published expected distances, and Ben's predecessor paths were verified. Each actual heap passed 18,000 randomised operations, and James's original heap test passed unchanged. This addresses the earlier uncertainty about agreement on the 27 datasets for this separate matched run, without altering the historical independent measurements.

Data Collection and Analysis
Benchmark results were automatically exported into CSV files, with each dataset producing a record containing its graph configuration, starting vertex and median execution time in microseconds (μs). Additional statistics were also recorded, including heap insertions, extractions, decrease-key operations, examined edges and successful relaxations. These operation counts were used to help identify differences in how the algorithms processed each graph.

Each dataset was identified by a filename containing its vertex count, edge count and random seed, allowing the results from both implementations to be matched and grouped by graph size and edge-to-vertex ratio. The three random seeds provided separate graph instances for each configuration, with the median execution times from these instances used to calculate a representative median for comparison.

For the matched rerun, execution times were compared using the Fibonacci-to-binary runtime ratio, calculated by dividing the Fibonacci implementation's median runtime by the binary implementation's median runtime. A ratio greater than one means that the Fibonacci implementation took longer. Ratios across the nine graph configurations use the median of each implementation's three

seed-specific medians. The original independent timings are retained as separate results and are not treated as a controlled speed comparison. Operation counts were examined separately to describe differences in workload and implementation behaviour; they do not by themselves establish the cause of the runtime difference.

Experimental Limitations

The experiments had several limitations regarding dataset size and graph structure. Only three graph sizes were tested: 250, 1,000 and 4,000 vertices, with edge-to-vertex ratios of $E = 4V$, $E = 16V$ and $E = 64V$. These configurations do not represent every possible graph size or structure. Although the configurations contain different numbers of edges, all three families remain asymptotically sparse because the number of edges is proportional to the number of vertices. Additionally, only three random seeds were used for each configuration, meaning that a larger sample could provide a more representative range of graph structures and decrease-key frequencies.

The original independent runs used different hardware, compiler settings and repetition counts, so those timings cannot isolate the effect of heap choice. The matched rerun addressed these measurement differences by compiling both actual implementations into one executable and using the same machine, graphs, source vertex and paired timing procedure. It retained all 432 raw batch measurements, 54 per-instance summaries and quartiles and interquartile ranges, allowing measurement variability to be examined. However, background load, CPU affinity, power mode and frequency scaling were not controlled, memory usage was not measured, and differences between the complete implementations still prevent attributing the runtime difference exclusively to the heap structure.

Results and Analysis

The final matched run compares James's binary-heap implementation with Ben's Fibonacci-heap implementation, using the same machine, graphs and timing harness. The binary implementation had the lower median on all 27 graph instances and all nine configuration aggregates. These results concern the complete submitted implementations; they are not a measurement of heap operations in isolation.

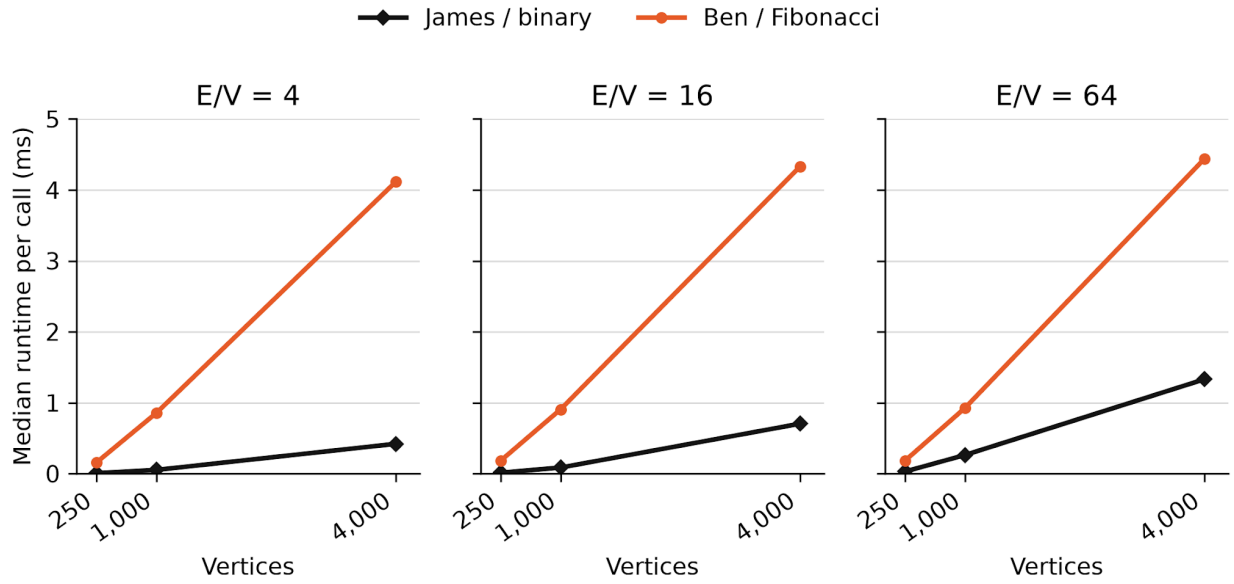


Figure 1. Matched Dijkstra runtime by vertex count and edge-to-vertex ratio. Each point is the median of three seed-specific medians; each seed-specific median summarises eight batch means. All panels use the same 0–5 ms vertical scale. Lines connect the measured configurations and do not represent additional measurements. Source: matched-run results/aggregate.csv, 4 November 2026.

At 250 vertices and E/V = 4, the aggregate runtime was 0.009092 ms for binary and 0.157444 ms for Fibonacci, giving a Fibonacci-to-binary ratio of 17.32. At 4,000 vertices, the corresponding runtimes were 0.421775 and 4.113850 ms at E/V = 4, 0.705425 and 4.329750 ms at E/V = 16, and 1.331150 and 4.438200 ms at E/V = 64. The ratios at that largest vertex count were therefore 9.75, 6.14 and 3.33 respectively.

For each tested vertex count, increasing the edge ratio reduced Fibonacci's relative disadvantage, although its absolute runtime remained higher. This is consistent with the prediction that edge-heavy workloads may make it more competitive, but it does not demonstrate a crossover. All tested families use E = cV for a fixed ratio c, so they remain asymptotically sparse. Substituting this relationship into the stated bounds leaves both algorithms with O(V log V) overall bounds for these families. These measurements cannot establish how either implementation behaves on genuinely dense families as V grows.

The operation counts also show why equal input graphs do not imply identical heap workloads. On the 4,000-vertex, 256,000-edge graph with seed 11, both implementations examined 256,000 edges. However, James recorded 10,820 decrease-key operations, while Ben recorded 14,814. James inserts a vertex when it is discovered, whereas Ben initially

inserts all vertices, so an initial discovery can count differently. Integer types, tie-breaking and Ben's predecessor output also differ.

Contiguous storage, allocation costs, pointer traversal and consolidation are plausible contributors to the measured gap. The present experiment does not isolate their individual effects. In particular, neither cache misses nor allocation time was directly profiled, so they should not be treated as proven explanations. The retained raw measurements and quartiles support further variability analysis, but the plotted seed medians have no confidence intervals. Three seeds per configuration and uncontrolled background activity also limit generalisation.

What Surprised Us

Ben identified the size of the performance gap between the two implementations as the main surprise. Fibonacci's better amortised decrease-key bound did not translate into lower complete-program runtime in this experiment. Its smallest aggregate disadvantage was still approximately 3.33 times, on the largest tested graph configuration, while the largest was approximately 17.32 times.

This result changes the practical interpretation of the theoretical comparison. An asymptotic advantage identifies an opportunity under a suitable workload; it does not specify the graph size at which a particular program will become faster. The narrowing relative gap is a reason to investigate further, rather than evidence that an unmeasured crossover must exist.

What We Would Do Next

The first follow-up would use one Dijkstra implementation with interchangeable heap backends. Both versions would use the same initialisation policy, integer types, tie-breaking and output contract. This would help distinguish heap choice from differences in the surrounding algorithms.

Further experiments would include larger graphs, genuinely dense families and graphs designed to produce many successful priority updates. Separate component measurements could examine allocation, consolidation and decrease-key costs. More seeds and repeated sessions under controlled CPU conditions would improve the evidence about variability. Memory should also be measured using one clearly defined metric for both implementations, rather than comparing heap-only storage with whole-process memory. These are proposed experiments, not completed results.

Individual Contributions

James was the primary author for the binary-heap implementation and its Dijkstra integration, standalone benchmarking and binary-heap checks. He supplied his repository

and README, contributed report material and AI-use disclosure, and recorded the binary-heap deep dive and first presentation segment.

Ben was responsible for the Fibonacci-heap side, with extensive AI assistance in code generation, research, testing and implementation revision as disclosed below. His work also covered the matched comparison, analysis and presentation preparation, including the slide material, recording script and final video assembly. He recorded the Fibonacci-heap deep dive and second presentation segment. The report distinguishes the independently developed implementations from the later shared-machine experiment.

AI Use Disclosure

AI Tools and Their Purpose

James:
Throughout this assignment, I took a different approach to using AI after reflecting on my experiences with the previous two assessments, particularly the feedback I received regarding AI usage in Assessment 1. This feedback made me more comfortable exploring the opportunities AI can provide when working on larger projects, and I chose to incorporate it into almost every stage of the assignment in the form of a personal tutor.

One of the main ways I used AI was to develop a project roadmap, code roadmap and report roadmap. By breaking the assignment into smaller tasks that I could progressively work through, I was able to keep track of my progress and ensure that I addressed all the marking criteria. This was particularly important to me because I had accidentally overlooked the “What I Learned” section in Assessment 1. Furthermore, by providing Ai with specific context about assignment requirements and my existing understanding, I was able to develop not only a project roadmap but also a learning roadmap this allowed me to build my understanding of Dijkstra’s algorithm step by step before attempting the full implementation, rather than relying on AI to product the complete solution.

I also found Ai particularly useful when structuring longer, text-heavy reports. My approach was to write initial drafts myself and then use AI to review the structure, improve the flow and identify technical inaccuracies. This helped me communicate my understanding more clearly while ensuring that the explanations remained technically accurate and reflected my own work.

Ben:
I used AI extensively to help with research, coding and implementation of the Fibonacci heap. It helped explain consolidation, decrease-key and cascading cuts, particularly how parent pointers, circular lists, degree counts and mark flags interact. I also used it to generate and revise implementation code and develop tests for heap operations and shortest-path results.

AI also helped with the experimental work, including integrating the two implementations into a shared benchmarking framework, collecting operation counts and analysing runtime differences. Its explanations helped connect the theoretical complexity bounds with the practical costs of allocation, pointer management and maintaining the heap structure.

AI Mistakes and Corrections

One of the biggest issues I encountered when using AI was the false sense of productivity it can create. The more AI is used, the more productive you can feel. However, sometimes that productivity is only a feeling, as you can make significant progress before stepping back and realising you don't fully understand the work you've created. I experienced this during the initial implementation of my binary heap, where I thought I was learning a lot by jumping straight into developing the complete solution. AI often reinforced this approach by providing encouragement and helping me move forward without necessarily challenging whether I understood what I was implementing. After recognising this early in development, I decided to scrap my initial implementation and create a code roadmap that would allow me to progressively learn and understand the core functionality of Dijkstra's algorithm using a binary heap before attempting the full assignment implementation.

Another challenge arose from how Ben and I initially approached the project. We had agreed on a shared CSV dataset format early in development, allowing us to independently build our implementations around the same graph data and work at our own pace. However, as development progressed, we realised that relying on AI to understand each other's algorithms had created a false sense of understanding. This became particularly obvious when we began drafting the report and found ourselves repeatedly asking each other questions about our respective implementations. Although AI could explain how the algorithms should work in theory, these explanations did not necessarily reflect how we had actually implemented them. To address this, we organised a meeting where we walked each other through our code and explained the decisions behind our implementations. This helped us develop a much clearer understanding of each other's work rather than relying on generalised AI explanations.

Ben:

One implementation issue arose in an earlier AI-generated Python prototype of the Fibonacci heap. The circular links between nodes could keep removed nodes alive until Python's garbage collector ran. Although the shortest-path results could still be correct, memory cleanup was delayed, which could also interfere with repeated timing measurements.

The issue was identified during code review and testing. The implementation was revised to clear retired nodes' links and ownership references, and explicitly release the remaining heap structure when a run ended. A regression test checked that nodes were released without waiting for cyclic garbage collection, including on disconnected graphs.

This highlighted a limitation of accepting working output as sufficient evidence of a correct implementation. With pointer-based structures, ownership and cleanup also need checking because they affect memory use and the reliability of performance measurements.

Impact on Group Collaboration

In terms of AI and group collaboration, many of the challenges have already been discussed in the previous section. However, one major limitation of AI that became particularly apparent during this assignment was its dependence on context. When working in a group with separate repositories, AI conversations and project documents, it can be difficult to use AI consistently across everyone's work, as the quality of its responses depends heavily on the information available to it.

For example, many programming issues can be resolved by progressively providing AI with additional context, such as error messages, previous attempts and relevant code. However, this becomes more difficult in a group environment when neither you nor the AI has a complete understanding of what your teammates have already attempted or the decisions they have made. This can lead to duplicated work, inconsistent suggestions or solutions based on incorrect assumptions. As a result, I believe that effectively using AI in group projects requires frequent communication and meetings to ensure everyone has an accurate understanding of the project's current state. Without this shared understanding, it becomes easy to provide AI with incomplete or outdated information and spend considerable time pursuing solutions that may not be appropriate for the actual implementation.